

Attempto Controlled Japanese (ACJ)

A Case Study with the `yamato-111` Parser

J. Kawasaki

July 24, 2025

Abstract

This paper introduces Attempto Controlled Japanese (ACJ), a subset of modern Japanese with a precisely defined and computable syntax. Drawing inspiration from Attempto Controlled English (ACE), ACJ is designed to be unambiguous and directly mappable to a formal representation. We present the complete lexical and syntactic specification of ACJ, which covers a range of fundamental sentence structures. We also describe the implementation of our recursive-descent LL(1) parser, `yamato-111`, in Go that translates ACJ sentences into an Abstract Syntax Tree (AST). This work represents a foundational step towards a formally verifiable and machine-processable subset of the Japanese language.

1 Introduction

Controlled Natural Languages (CNLs) are subsets of natural languages that are restricted in a systematic way to reduce ambiguity and complexity. Projects like Attempto Controlled English (ACE) have demonstrated that CNLs can serve as a bridge between human language and formal logic, enabling precise and verifiable specifications.

This paper presents a similar effort for the Japanese language. We define a subset of Japanese, which we call Attempto Controlled Japanese (ACJ), and provide a formal grammar for it. The main contribution of this work is to define a machine-executable grammar and provide a reference implementation of a parser, which translates ACJ text into an unambiguous Abstract Syntax Tree (AST).

2 The Attempto Controlled Japanese (ACJ) Language

ACJ is defined by a lexicon and a set of construction rules.

2.1 Lexicon

The lexicon of ACJ consists of predefined tokens for particles, auxiliaries, and a copula, as well as user-defined content words (nouns, verbs, adjectives).

- Nouns, Verbs, Adjectives: Word stems (e.g., ”学生”, ”食べ”).
- Case Particles: が, を, に, は, の.
- Auxiliary Verbs (AUX): ます, た.
- Copula (COPULA): です.

2.2 Syntactic Rules

The grammar is specified in an ANTLR-like notation and is designed to be LL(1).

```
sentence: clause+;
clause: np vp;
np: NOUN caseParticle?;
vp: (np)? (verbPhrase | copulaPhrase);
verbPhrase: VERB auxList;
copulaPhrase: (NOUN | ADJ) COPULA;
auxList: AUX auxList | /* epsilon */;
```

3 Parser Implementation

We implemented a recursive-descent parser for ACJ in the Go programming language. The parser, named `yamato-111`, takes a string of ACJ text and, if the string conforms to the grammar, returns an Abstract Syntax Tree (AST). The main components are a lexer that performs tokenization and a parser that builds the AST according to the syntactic rules.

4 Conclusion and Future Work

We have defined Attempto Controlled Japanese (ACJ), a machine-processable subset of Japanese, and provided a reference parser. The resulting ASTs provide a solid foundation for further formal processing.

Future work will focus on expanding the grammatical coverage of ACJ and, crucially, defining a direct mapping from ACJ’s ASTs to a formal logic representation, such as First-Order Logic, similar to the approach taken by ACE.